Portfolio

빠르고 안정적인 서비스를 지향합니다.
백엔드 개발자 **김예찬** 입니다.

About Me

저를 소개합니다

김예찬 **Kim Yechan**

백엔드 개발자

010-4936-5651

yechan1601.dev@gmail.com

1998.01.16

https://constant1601.tistory.com

https://github.com/macmorning0116

빠르고 안정적인 서비스를 지향합니다.

빠르고 안정적인 서비스를 지향하는 개발자 김예찬입니다. 병목 지점을 분석해 성능을 개선하고, 개선 결과를 수치로 검증하며, 지속적인 모니터링을 통해 신뢰할 수 있는 서비스를 만듭니다.

Project 01

buptle

BuptleBiz (주) 법틀

2025.07 ~

주요목표 기업의 법무를 하나의 통합시스템에서 관리

핵심기여

- ✔ AKS 환경 보안성 개선
- ✔ 통계 페이지 API 응답 시간 개선
- ✔ 웹 에디터 XSS 취약점 분석 및 개선
- ✔ 신규 프로젝트 API 개발 및 레거시 코드 리팩토링

사용기술 Python, Django, Docker, Azure Kubernetes Service

BuptleBiz

AKS(Azure k8s Service) 보안성 개선

문제상황

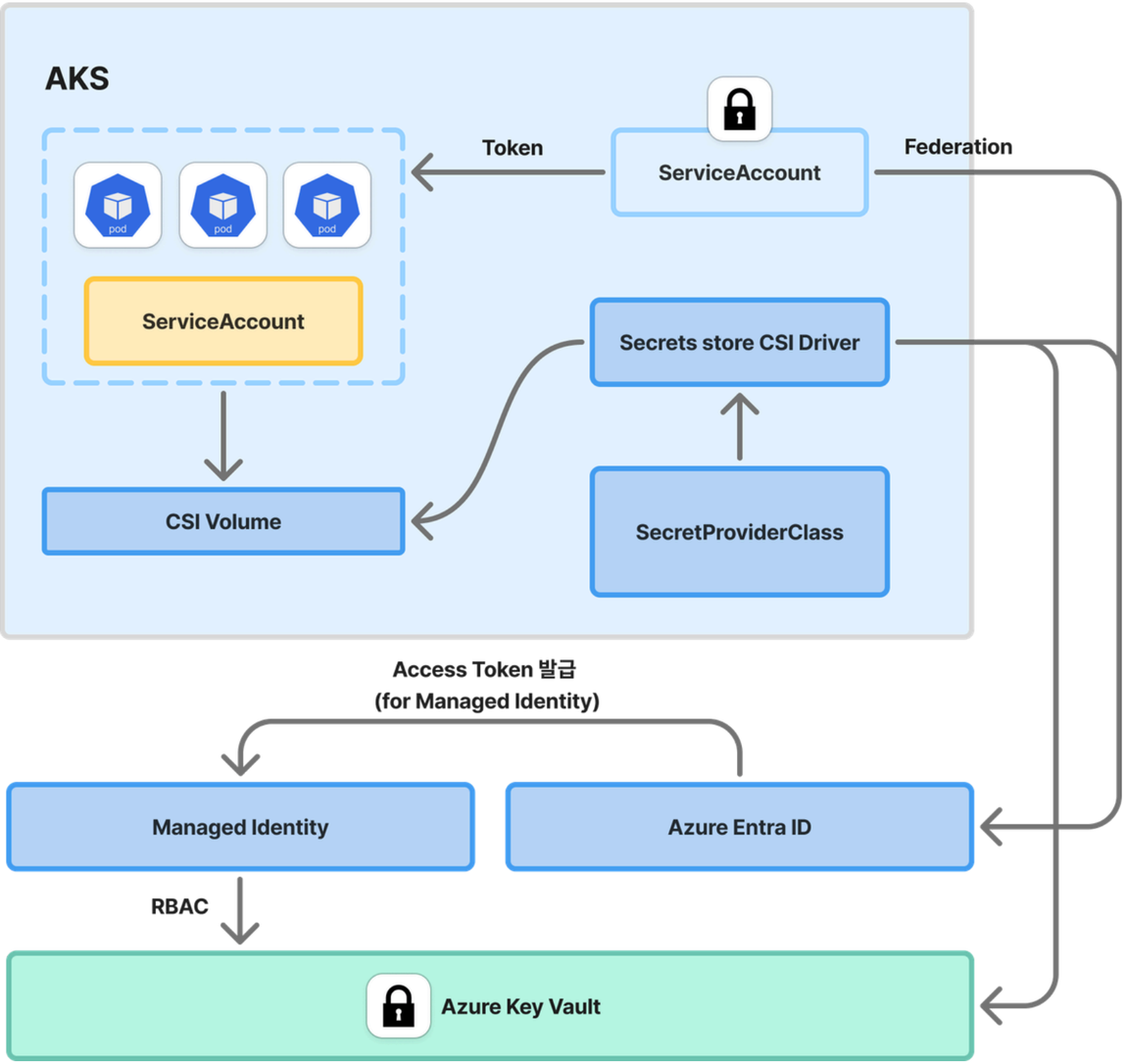
기존에는 AKS에서 DB 계정·외부 API Key·인증서 같은 민감정보를 Kubernetes Secret으로 관리하고 있었습니다. 하지만 법무 시스템 특성상 보안 요구 수준이 높았고, Secret이 클러스터 리소스로 저장되다 보니 접근 통제·감사·로테이션을 일관되게 적용하기 어려웠습니다.

문제 해결

이를 해결하기 위해 민감정보 관리를 Azure Key Vault 중심으로 전환했습니다. AKS에는 Secrets Store CSI Driver를 적용하고, SecretProviderClass로 환경별 참조 대상을 선언적으로 관리했습니다. 이후 Pod가 시작될 때 CSI Driver가 Azure Key Vault에서 필요한 Secret을 조회해 컨테이너 파일 시스템에 볼륨 형태로 마운트하도록 구성했습니다. 또한 Azure Entra ID 기반의 인증 체계를 통해 Key Vault 접근 권한을 안전하게 제어했습니다. 추가로 컨테이너 하드닝을 위해 runAsNonRoot, readOnlyRootFilesystem를 적용하고 privileged를 비활성화했습니다.

결과

고객사 보안점검(Secret 관리, 권한 통제, 컨테이너 권한 제어)을 충족하고,
최종 평가에서 우수 보안 등급(주요 지적 0건)을 달성하였습니다.



BuptleBiz

통계 페이지 API 응답 시간 개선

문제상황

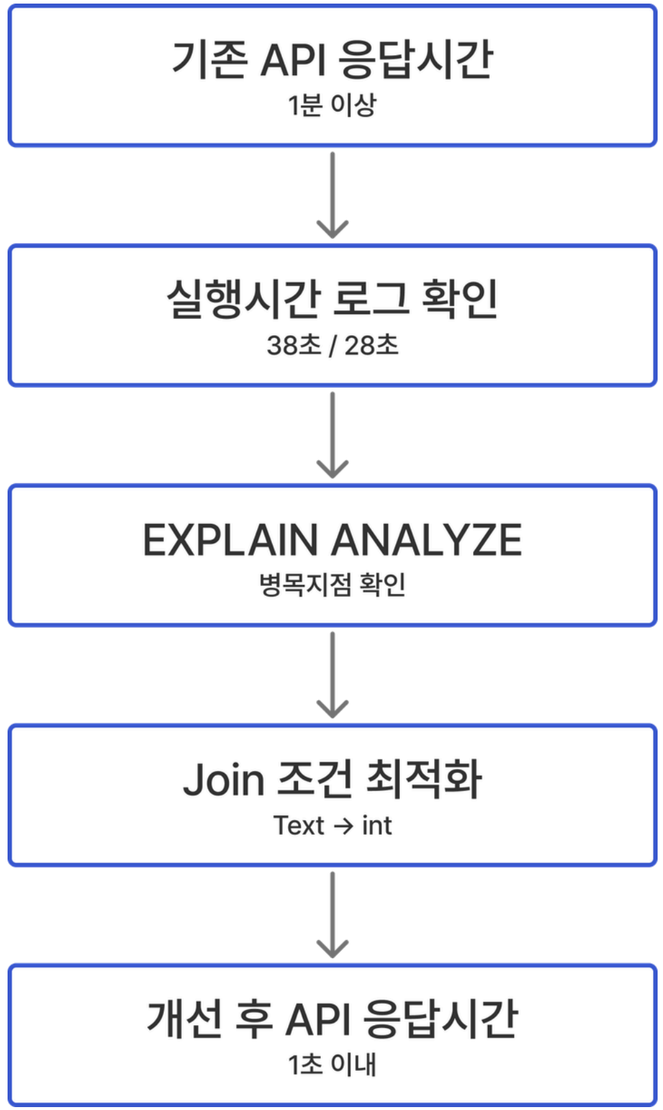
통계 페이지 API 응답이 1분 이상 소요되는 문제가 있었습니다. 해당 페이지는 진입 시 약 15건의 쿼리가 실행되는 복잡한 구조였기 때문에, 실제 병목 지점을 정확히 파악한 뒤 개선하는 작업이 필요했습니다.

문제 해결

우선 애플리케이션 단에서 메서드별 실행 시간 로그를 추가해 구간별 성능을 측정했습니다. 그 결과, 쿼리를 수행하는 2개의 메서드에서 각각 38초, 28초가 소요되고 있음을 확인했습니다. 이후 cursor.mogrify(sql, params)를 활용해 실제 실행 SQL을 추출하고, EXPLAIN ANALYZE 기반으로 실행 계획을 분석했습니다. 분석 결과 2개의 쿼리 모두main_userprofile과 history를 조인하는 구간에서 대규모 Nested Loop가 발생하고 있었으며, Rows Removed by Join Filter: 116,950,912가 확인되었습니다. 조인 대상 컬럼이 PK 기반이라 인덱스를 활용할 수 있는 구조였음에도 불구하고, 양쪽 값을 text로 변환하여 비교하고 있어 인덱스를 활용하지 못하고 있었습니다.이에 조인 조건을 인덱스를 활용할 수 있는 형태로 수정하여, 실행 계획이 더 효율적으로 동작하도록 개선했습니다.

결과

counsel_review_data : 36043.160ms → 12.446ms
contract_review_data : 27496.683ms → 13.398ms
API 응답 시간 1분 이상 → 1초 이내로 약 98.5% 이상 개선되었습니다.



BuptleBiz

웹 에디터 XSS 취약점 분석 및 개선

문제상황

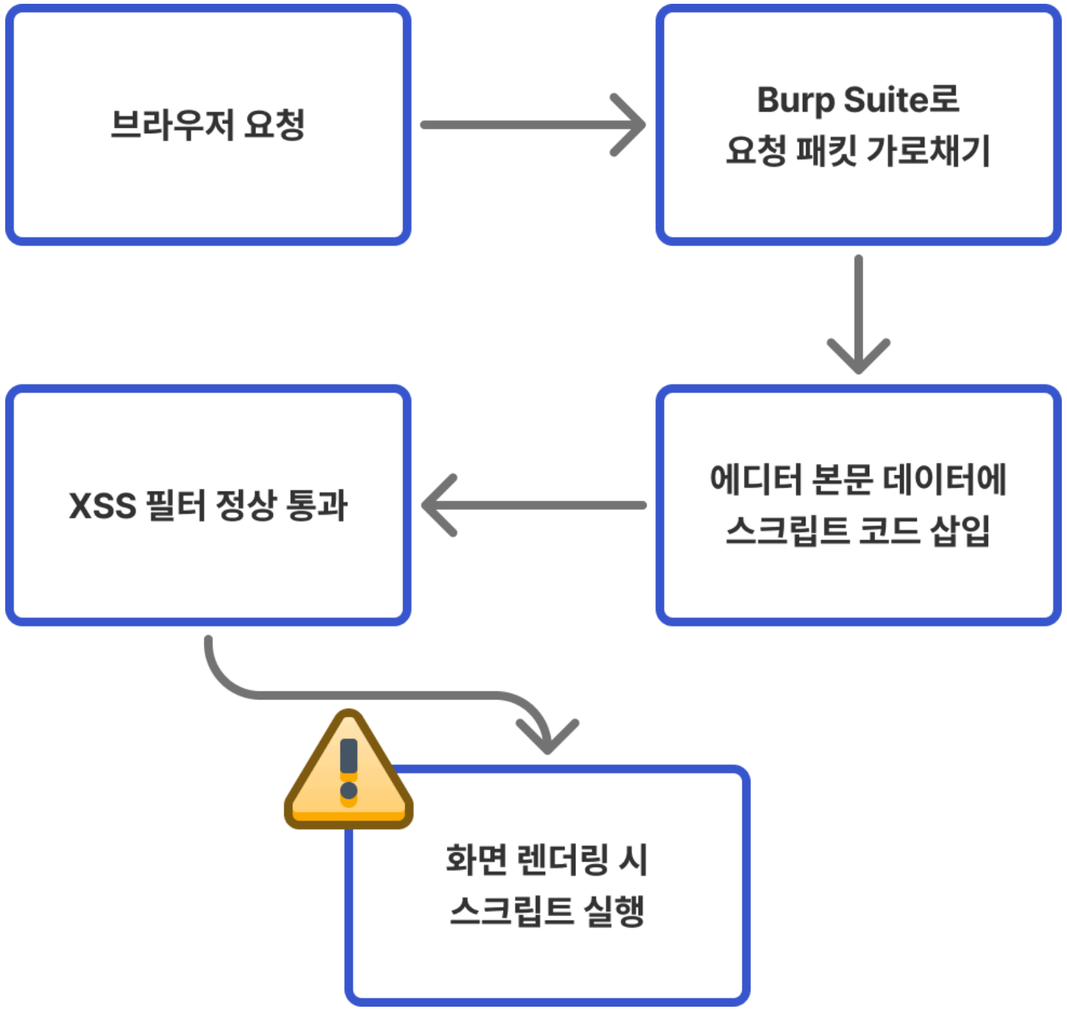
자사 서비스의 웹 에디터 기능에 XSS(Cross-site Scripting) 취약점이 발견되었습니다. 해당 취약점은 스크립트가 저장/실행 될 수 있었기에 중요한 보안 이슈였습니다.

문제 해결

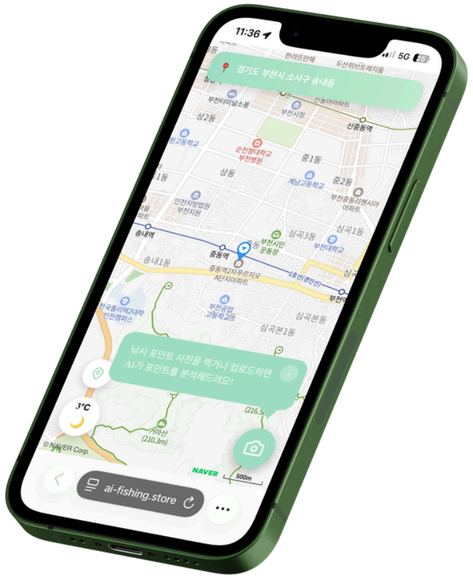
정상적인 웹 에디터 입력 요청에서는 <, > 문자가 <, > 형태로 치환되어 전달되고 있어 현상 재현이 불가능했습니다. 이후 DB 데이터에 직접 스크립트 코드를 삽입하여 화면 렌더링 시 스크립트가 실행되는 것을 확인하였습니다. 이를 통해 입력 단계 자체보다는 요청 전송 과정에서 데이터가 변조되었을 가능성을 의심했습니다. 가설 검증을 위해 Burp Suite(웹 프록시 기반 보안 테스트 도구)로 브라우저와 서버 사이 요청 패킷을 가로채 에디터 본문 데이터를 수정한 뒤 재전송 했고, 스크립트가 그대로 저장/실행 되는것을 재현할 수 있었습니다. 이후 서버 측 XSS 필터 코드를 분석한 결과, 웹 에디터 본문 파라미터인 document_html이 예외 처리 목록에 포함되어 검사 대상에서 제외되고 있음을 확인했습니다. 해당 예외 처리는 현재 코드상 별도의 참조처나 유지 필요성이 확인되지 않았고, 과거 호환성 목적으로 남아 있던 불필요한 코드로 판단되어 제거하였습니다.

결과

웹 에디터의 요청 또한 일반 요청과 동일하게 XSS 필터를 적용하도록 개선하였습니다.
이후 보안 점검에서 XSS 취약점이 해소되었음을 확인하였습니다.



Project 02



Ai-Fishing

2025. 12 ~ 2026.01

주요목표

AI를 활용한 낚시 포인트 추천 1인 프로젝트

핵심기여

- ✔ 운영 비용 최적화를 위한 로그/메트릭 관측 시스템 재설계
- ✔ CI/CD 파이프라인 구축(Git Actions + Docker + GHCR + EC2)
- ✔ 주요 로직 테스트 코드 작성(Line Coverage 100%)

사용기술

Java, SpringBoot, AWS, Docker, Grafana Cloud, Junit5
TypeScript, React

https://github.com/macmorning0116/my-fishing-server
https://www.ai-fishing.store

Ai-Fishing

운영 비용 최적화를 위한 로그/메트릭 관측 시스템 재설계

문제상황

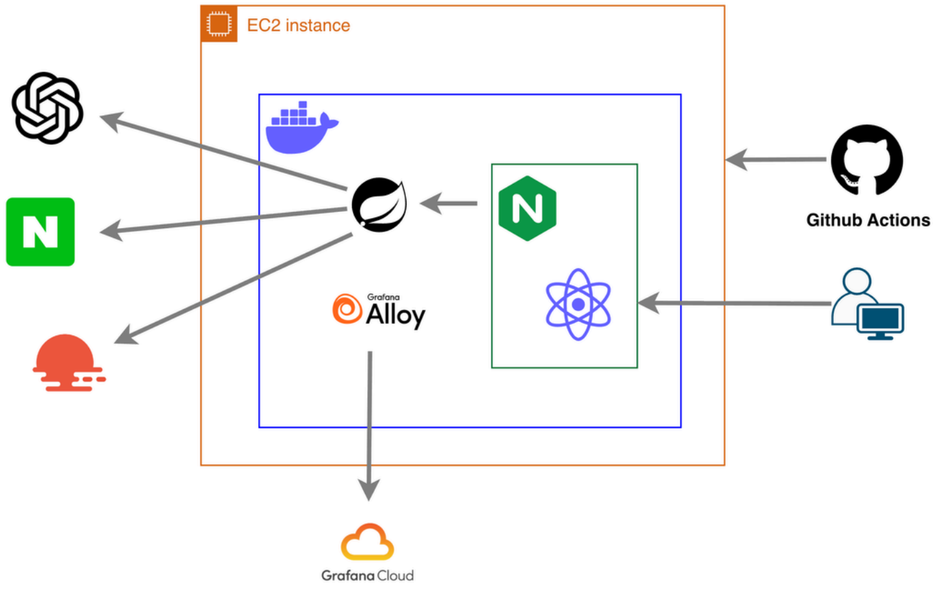
EC2 t3.micro(1GB RAM) 하나에서 Nginx/Spring Boot뿐 아니라 Prometheus·Loki·Grafana·Alertmanager까지 Docker로 함께 운영하고 있었습니다. 이 구조에서 관측 스택이 메모리를 지속적으로 점유하면서 여유 메모리가 거의 남지 않았고, 트래픽이 몰리는 순간 OOM(Out Of Memory)이 발생하였습니다.

문제 해결

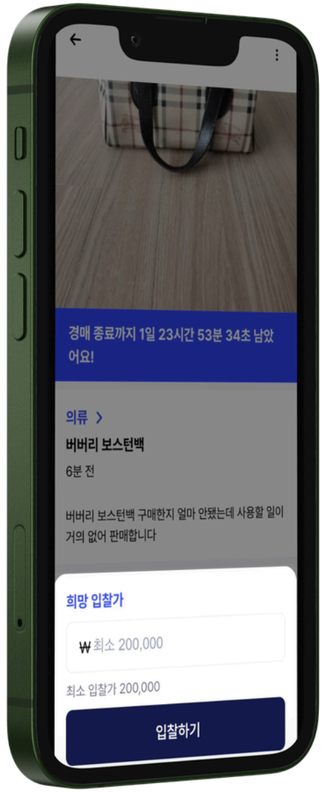
단순히 인스턴스를 t3.small로 올리면 안정성은 확보되지만 월 고정비가 증가하고, swap 메모리를 사용하면 OOM 회피책은 될 수 있지만 근본 해결은 아니기에, 작은 인스턴스에서 IO 병목으로 지연 시간을 키울 우려가 크다고 판단했습니다. 따라서 관측스택을 EC2 내부에서 직접 운영하는 방식이 아닌 Grafana Cloud(Free tier)로 로그/메트릭/알림을 이관하는 방향으로 재설계했습니다. EC2에는 Grafana Alloy + node_exporter만 남겨서 애플리케이션/호스트 지표를 수집한 뒤, Grafana Cloud로 전송하도록 구성했고(원격 저장/쿼리/대시보드/알림은 Cloud에서 처리), 기존에 메모리를 크게 소모하던 Prometheus·Loki·Grafana·Alertmanager 컨테이너는 제거했습니다.

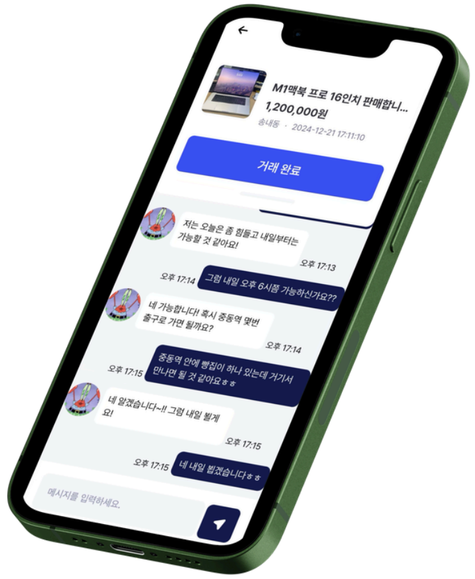
결과

관측 스택 컨테이너를 제거하면서 t3.micro 사양을 유지하고도 OOM이 해소되었습니다. 고정 지출을 월 $18.98에서 $0으로 절감하였습니다.



Project 03





Meerket

2024. 11 ~ 2025.01

주요목표

동네기반 중고거래 서비스에 입찰 시스템을 도입하여 판매자와 구매자 모두 합리적인 가격으로 거래할 수 있도록 하는 플랫폼

핵심기여

- ✔ 동시성과 중복 방지(따닥 이슈)를 고려한 입찰 시스템 설계 및 개발
- ✔ 서비스 장애 복구 및 입찰 마감 자동화 프로세스 구현
- ✔ 실시간 채팅 서비스 설계 및 구현(STOMP, mongoDB, redis, FCM)

사용기술

Java, SpringBoot, AWS, Docker, Junit5, MySQL, Redis, MongoDB

https://github.com/J1P5/Meerket__BE

Meerket

Quartz블로그 정리 링크

장애복구가 가능한 입찰 시스템 설계 및 구현

문제상황

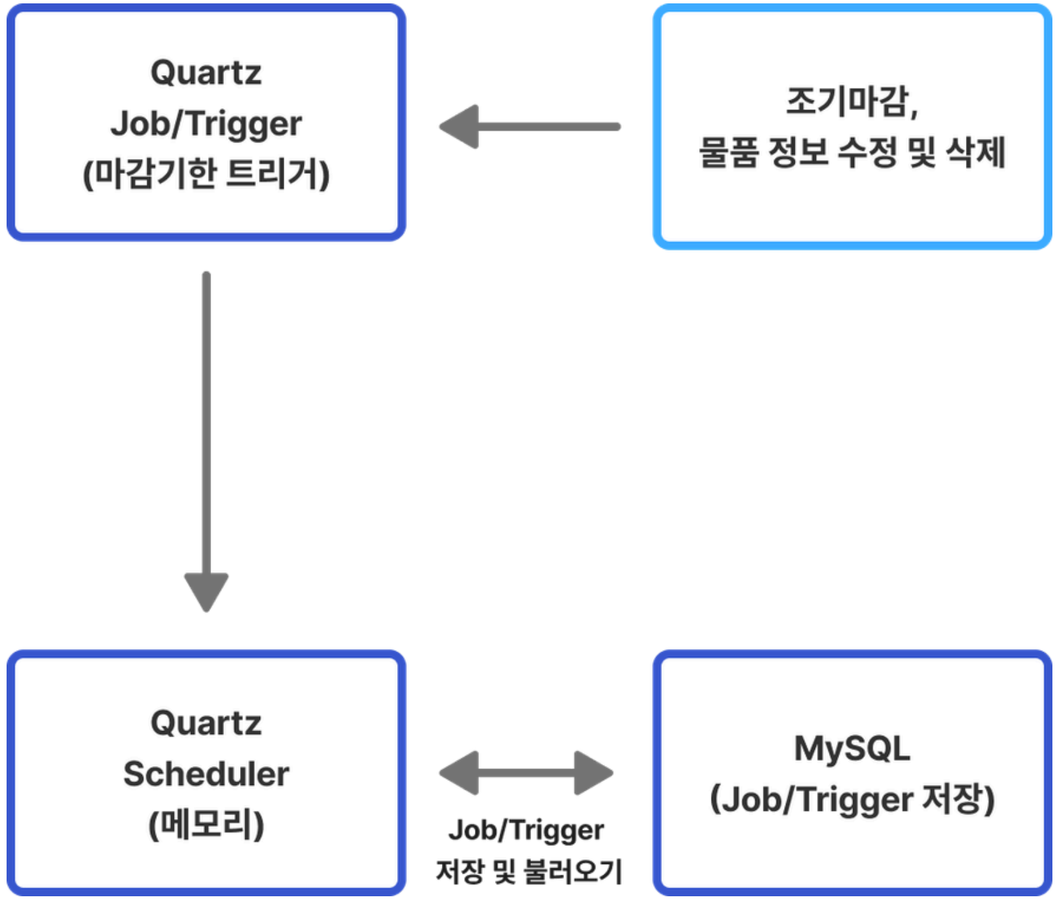
입찰 시스템을 구현하면서 동일 사용자의 연속 클릭(따닥)이나 네트워크 재전송으로 인해 중복 요청이 발생할 수 있었고, 판매자와 구매자 요청에 동시성 제어가 필수적이었습니다. 또한 입찰에는 마감이라는 시간 제약이 있어, 서비스 장애나 배포 등으로 서버가 재시작되더라도 마감 스케줄 정보가 유실되지 않고 그대로 유지되어야 했습니다.

문제 해결

중복 요청(따닥) 방지를 위해 Redis의 SETNX + TTL을 활용해 일정 시간 내 동일 요청은 처리되지 않도록 제어했습니다. 동시성 제어는 Redis 기반 RLock을 적용해 임계 구간(입찰 반영/상태 변경)을 보호하고, 경쟁 상황에서도 DB 정합성이 깨지지 않도록 처리했습니다.
입찰 마감은 상품마다 마감 시간이 달라 동적으로 스케줄링되어야 했고, 서버 재시작에도 마감 정보가 유지되어야 했기 때문에 Quartz 스케줄러를 도입했습니다. 스프링 기본 스케줄러는 스케줄을 유연하게 관리하기 어렵고, 재기동 시 스케줄 상태를 직접 영속화·복구해야 하는 부담이 있어, Quartz의 Job/Trigger 모델과 영속 저장(JobStore)을 활용해 스케줄을 DB에 저장하고 재시작 시 자동 복구되도록 구성했습니다.

결과

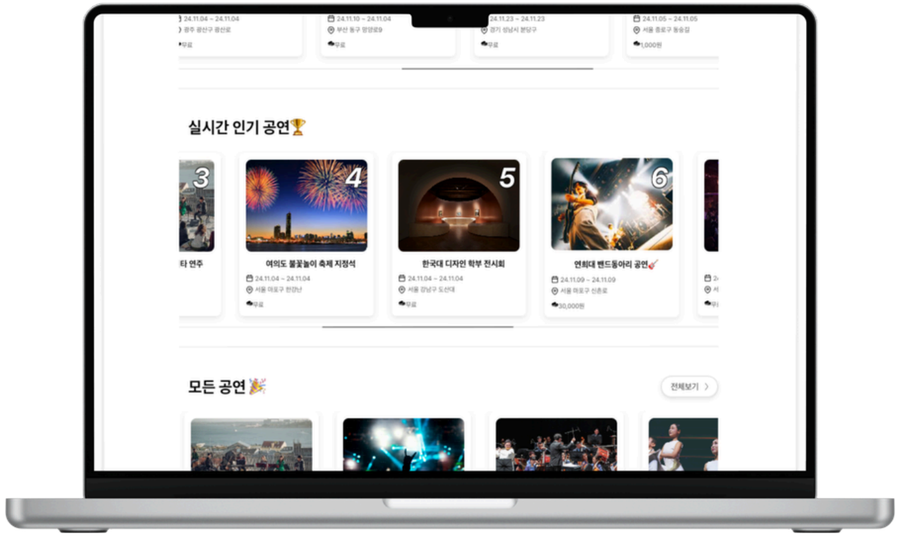
JMeter로 동시 요청 100건을 발생시켰을 때 1건만 정상 처리되었습니다.
JUnit기반 동시성 요청 100건 중 DB에 잘못 삽입된 값 0건으로 정합성을 확인했습니다.
서버를 고의로 재실행하는 장애 시나리오에서도 마감 정보 유실 0건을 확인하여 장애 상황에서도 일관된 입찰 마감 처리를 보장했습니다.



Project 04

Somun

2024. 09 ~ 2024.11



주요목표

동아리, 버스킹 같은 소규모 공연을 개최하거나 관람하고자 희망하는 이용자들을 위한 서비스

핵심기여

- ✔ 캐싱과 스케줄러를 활용한 성능 개선
- ✔ SQL 쿼리 최적화를 통한 성능 개선
- ✔ Embedded Redis를 도입한 로컬 개발환경 구성

사용기술

Java, SpringBoot, MySQL, Redis
JavaScript, React

https://github.com/prgrms-be-devcourse/NBE1_2_Team05

Somun

캐싱과 스케줄러, 쿼리 튜닝을 통한 성능개선

문제상황

최근 인기 공연 조회의 초기 구현에서는 요청이 들어올 때마다 메인 DB를 직접 조회하는 구조였기에, 동일한 결과를 반복적으로 DB에서 조회하면서 불필요한 부하가 누적되었습니다. 맞춤 공연 추천은 다중 JOIN 결과에 대해 COUNT(DISTINCT)와 GROUP BY, ORDER BY가 한 번에 수행되며 실행 비용이 커져 응답 지연과 DB 부하가 발생했습니다.

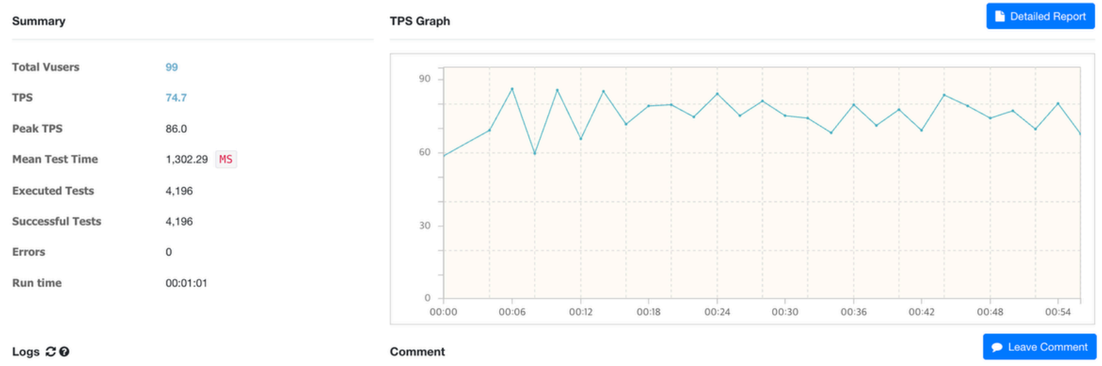
문제 해결

인기 공연 조회는 Redis 캐싱 + 스케줄러 기반 갱신 구조로 개선했습니다. 스케줄러가 주기적으로 메인 DB에서 인기 공연 데이터를 조회해 Redis에 캐싱하고, 요청 시에는 Redis에 캐시된 데이터의 최신 여부를 확인한 뒤 캐시가 유효하면 Redis에서 즉시 응답하도록 구성했습니다. 캐시가 없거나 갱신이 필요한 경우에만 DB를 조회하고 캐시를 업데이트하여, 반복 조회로 인한 DB 부하를 근본적으로 줄였습니다.

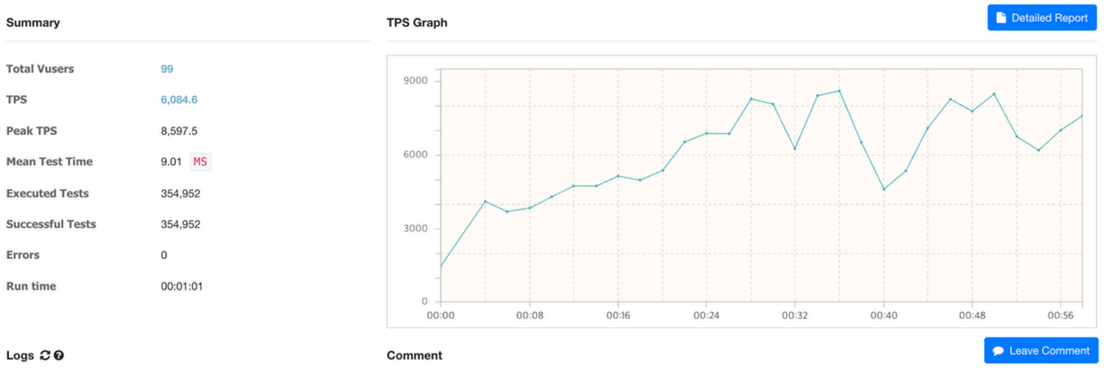
추천 공연 조회는 기존 쿼리의 병목(다중 JOIN 이후 COUNT(DISTINCT)+GROUP BY+ORDER BY)을 줄이기 위해, 공연별 카테고리 매칭 수를 먼저 계산하는 서브쿼리로 범위를 축소하고, 상위 N개만 추린 뒤에 조인하여 상세 정보를 가져오도록 쿼리를 재구성했습니다.

결과

nGrinder 부하 테스트 기준으로 인기 공연 조회는 TPS 74.7 → 6084.6, 평균 응답 시간은 1302ms → 9ms, 추천 공연 조회 쿼리는 데이터 7만건 기준 평균 실행 시간이 200ms → 70ms(약 65%)로 개선되었습니다.



개선 전 부하테스트 결과



개선 후 부하테스트 결과

Thank you

제 포트폴리오를 봐주셔서 감사합니다.